

SYSTEM DESIGN DOCUMENT

HERALD

Universal Agentic Interface Layer

Human-to-machine translation

Self-improving

Cross-environment

Open source · MIT

• What is HERALD?

HERALD is the central orchestrator for all agentic task execution. It is the only agent with cross-system awareness. Every other agent has a single defined scope and communicates exclusively with HERALD — no agent communicates with another agent directly.

HERALD sits between human input and every downstream agent. It runs discovery, manages planning, controls the agent lifecycle, dispatches work, tracks execution state, and learns from every execution.

6

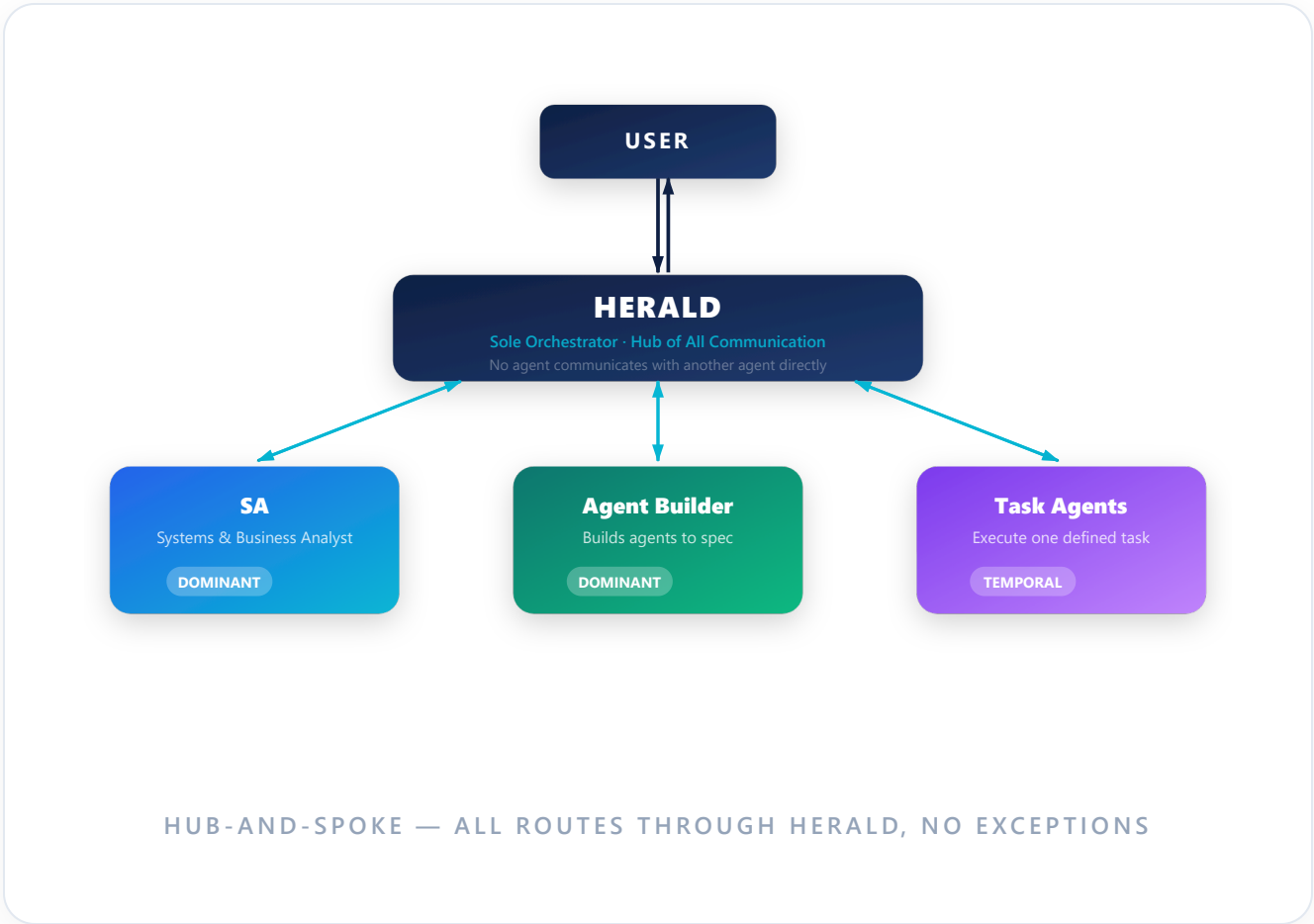
95%

3

∞

● **Architecture**

HERALD operates as a hub-and-spoke orchestrator. All communication routes through HERALD. No exceptions — no agent ever contacts another agent directly.



SA
DOMINANT
Validate specs, plan execution, classify agents needed, score outcomes

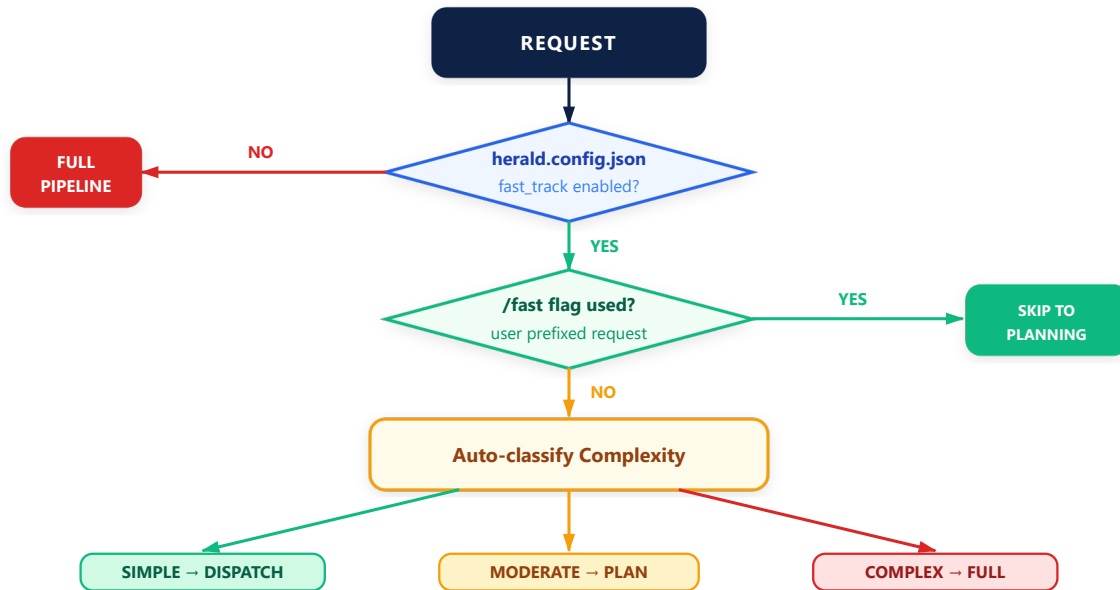
Agent Builder
DOMINANT
Build new agents to spec — purpose, scope, spawn type, and instructions

Task Agents
TEMPORAL OR DOMINANT
Execute one defined task. Nothing else. Scope never

expands.

• Fast-Track Mode

Three stacked mechanisms to reduce pipeline overhead for low-risk or time-sensitive work. Project config always takes precedence.



FAST-TRACK DECISION TREE — CONFIG OVERRIDES ALL



Config Override

herald.config.json — set **fast_track: false** to enforce full pipeline on critical projects. Overrides all flags and classification.



/fast Flag

User prefixes request with **/fast** to skip discovery and context loading. Jumps straight to SA planning. Plan approval is never bypassed.



Auto-classification

HERALD classifies every request automatically — **Simple, Moderate, or Complex** — and applies the appropriate pipeline level.

LEVEL	CRITERIA	PIPELINE APPLIED
Simple	Single file · No ambiguity · No new agents · No cross-system impact · Low risk	Skip to dispatch
Moderate	Multiple files · Some ambiguity · Existing agents sufficient · Limited cross-system impact	Skip discovery, run planning + approval
Complex	New agents needed · Cross-system · High risk · Unclear intent · Multiple viable approaches	Full pipeline

• The Six Layers



SIX-LAYER PIPELINE · LEFT TO RIGHT · FULL OR FAST-TRACKED

1 INTENT

Intent Engine

- Check `herald.config.json` — apply fast-track rules
- If `/fast` used and enabled → skip to layer 3
- Classify complexity: simple, moderate, or complex
- Full discovery session across all 8 dimensions: Intent · Scope · Constraints · Stack · Format · Timeline · Dependencies · Risk tolerance
- Do not proceed until all dimensions are answered or confirmed not applicable

2 CONTEXT

Context Harvester

- Check `plans/` for any plan with status: `in_progress`
- If found — surface to user with live checklist and offer to resume from last incomplete item
- Otherwise load only what is relevant: files, schemas, configs, prior decisions
- Output a concise context summary: what was found, where it lives, what matters

3 PLAN

Plan Architect

HERALD DISPATCHES SA

- SA analyzes project structure and checks `agent-registry.json`

- SA classifies each required agent as **temporal** or **dominant**
- SA produces viable plans → HERALD presents each with Approach · Pros · Cons · Risks
- HERALD waits for user to select a plan — never proceeds without approval
- Approved plan saved to plans/ with a full checklist (all items: pending)



SYNTHESIZE

Prompt Synthesizer

- HERALD writes a precise brief for each agent: objective, context, constraints, output spec, KB patterns
- Context scoped to each agent only — nothing extra
- No raw user input is ever passed to an agent



DISPATCH

Dispatch Router

HERALD DISPATCHES AGENT BUILDER IF NEEDED

- Compare plan requirements against agent-registry.json
- Missing agents → Agent Builder dispatched with full spec; new dominant agents registered
- Execute in order — sequential or parallel per approved plan
- Each completed task updates the checklist: status · timestamp · retries
- Temporal agents closed on task completion



FEEDBACK

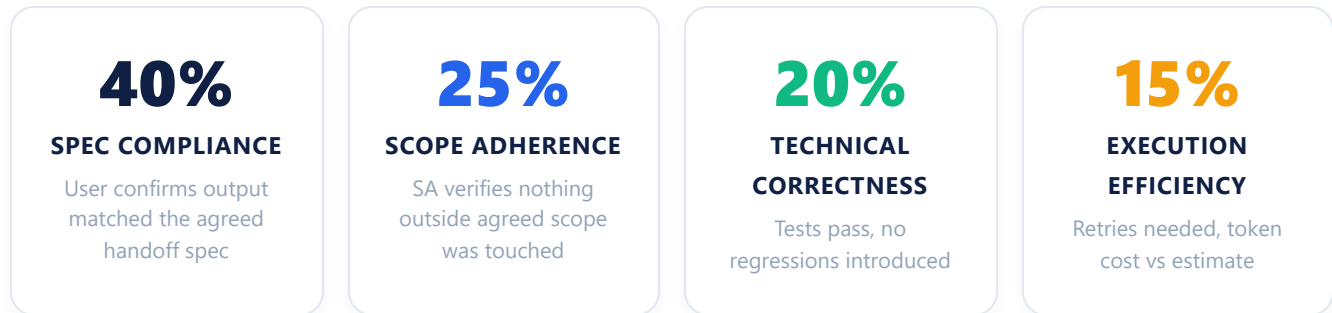
Feedback Loop

HERALD DISPATCHES SA

- Plan marked completed · SA dispatched to score
- SA asks user: *"Does the output match what was agreed in the handoff spec?"*
- Composite score calculated → reported to HERALD → written to plan file
- **≥ 95% → pattern stored in knowledge-base.json**
- **< 95% → failure dimensions tagged, no pattern stored**

• Success Scorecard

Every execution is scored by the SA across four weighted dimensions. The composite score determines whether the pattern is stored in the knowledge base.



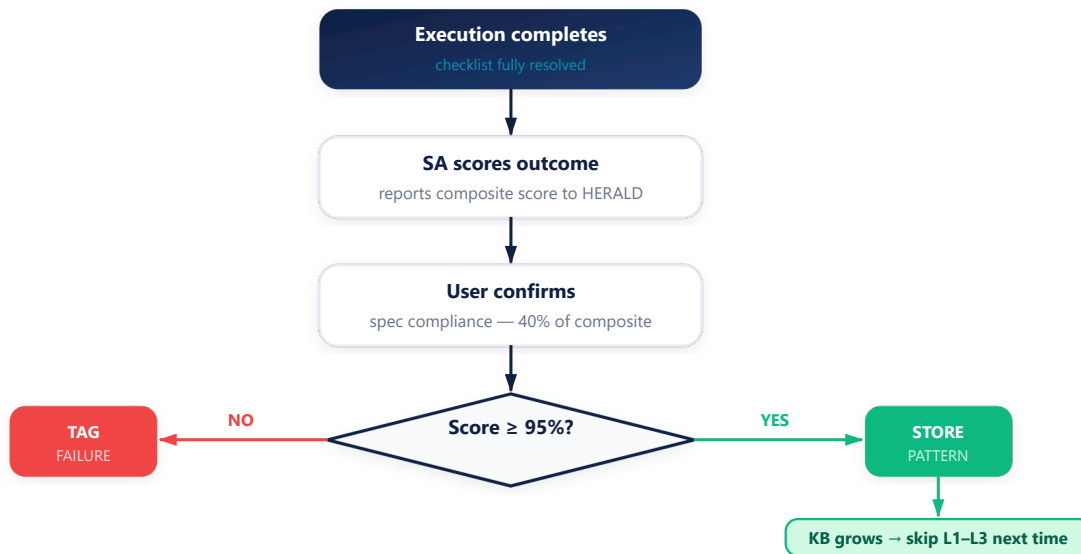
95%

Composite threshold to store a pattern

Scores below 95% tag failure dimensions and record what fell short — but no pattern is stored. This keeps the knowledge base clean and high-quality over time.

• Self-Improvement Flywheel

HERALD gets smarter with every successful execution. Stored patterns let HERALD skip layers 1–3 on matched requests — making repeat task types faster and cheaper over time.



SELF-IMPROVEMENT FLYWHEEL — PATTERNS ACCUMULATE, PIPELINE GETS FASTER

Rules

HERALD IS THE SOLE ORCHESTRATOR

All agent communication routes through HERALD. No agent talks to another agent directly.

SINGLE SCOPE

Every agent does exactly one thing. Scope is defined at creation and does not expand.

NO RAW PASSTHROUGH

HERALD never passes raw user input to downstream agents. All input is translated first.

PLAN APPROVAL GATE

HERALD never dispatches without the user approving a plan first. Always.

PLANS ARE PERSISTENT

Every approved plan is saved with a live checklist. HERALD resumes in-progress plans on reinitialization.

CONFIG IS KING

`herald.config.json` overrides all flags and auto-classification. Lock down critical projects here.

QUALITY GATE

Composite score $\geq 95\%$
required to store a pattern.
Keeps the knowledge base
clean and trustworthy.

AGENT LIFECYCLE

HERALD spawns temporal
agents and closes them.
HERALD registers and
maintains dominant agents.

**ENVIRONMENT-
AGNOSTIC**

HERALD operates identically
regardless of the downstream
environment — codebase,
API, ERP, or any other system.

• File Schemas

plans/2026-03-17_task-name.json

```
{
  "id": "2026-03-17_jwt-auth-express",
  "created": "2026-03-17T10:00:00",
  "complexity": "simple | moderate | complex",
  "fast_track": false,
  "status": "pending | in_progress | completed | failed",
  "approach": "summary of the approved plan",
  "checklist": [
    {
      "task": "description of the task",
      "agent": "agent_id",
      "spawn_type": "temporal | dominant",
      "status": "pending | in_progress | completed | failed",
      "started_at": null,
      "completed_at": null,
      "retries": 0
    }
  ],
  "score": {
    "composite": null, "spec_compliance": null,
    "scope_adherence": null, "technical_correctness": null, "execution_efficiency": null
  },
  "pattern_stored": false
}
```

 herald.config.json

```
{
  "fast_track": {
    "enabled": true,
    "allow_slash_fast": true,
    "auto_classify": true
  },
  "pipeline": {
    "require_plan_approval": true,
    "success_threshold": 95
  },
  "version": "1.0"
}
```

 knowledge-base.json

```
{
  "patterns": [
    {
      "input_pattern": "type of request",
      "engineered_prompt": "the brief that worked",
      "agents_used": ["agent_a", "agent_b"],
      "plan_id": "2026-03-17_task-name",
      "score": {
        "composite": 95, "spec_compliance": 95,
        "scope_adherence": 100, "technical_correctness": 95, "execution_efficiency":
      },
      "retries": 0, "token_cost": 420
    }
  ],
  "version": "1.0", "last_updated": null
}
```

HERALD · Universal Agentic Interface Layer · MIT License · Built to make AI coding accessible to everyone